

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("practice03.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

Practice 03 — Reshaping and Joining Real-World Data

Adapted from an original practice by Prof. Ken Arnold.

In this practice you will work with the real [Gapminder \(https://www.gapminder.org/data/\)](https://www.gapminder.org/data/) dataset — country-level GDP and life expectancy, tracked across decades. Each task is tagged with the SLO it covers:

SLO	Description
04A	Describe the structure of relational data and identify key columns and relationships between tables
04B	Join tables using different join types (inner, left, right, outer) and explain when each is appropriate
04C	Reshape data between wide and long (tidy) formats

The Dataset: Gapminder



[Gapminder \(https://www.gapminder.org/data/\)](https://www.gapminder.org/data/) is a Swedish foundation whose goal is to "promote sustainable global development through the use of data and statistics." It's best known for the animated bubble charts that made Hans Rosling famous — the same charts you'll build at the end of this practice.

Ordinarily, you'd visit [gapminder.org](https://www.gapminder.org), search for an indicator, and download a spreadsheet yourself. To keep this practice self-contained and reproducible, we've done that step for you: `plotly` — the same library you've used all semester — ships with a snapshot of this exact dataset built in `(px.data.gapminder())`, covering 142 countries from 1952 to 2007 in 5-year steps. Nothing to download.

```
In [ ]: import pandas as pd
import plotly.express as px
```

Setting Up Today's Tables (given — you don't need to modify this)

The cell below builds two **wide** tables — `gdp_wide` and `life_wide`, one row per country, one column per year — formatted the way Gapminder's own website actually exports its data. Notice `gdp_wide` uses a "k" suffix for thousands (e.g. "5.9k" for \$5,900 — a real quirk of Gapminder's downloads, not something we added). It also builds `country_region`, a small lookup table of each country's continent. Run it and look at the results — you don't need to modify any of this.

```
In [ ]: _gm = px.data.gapminder()

def _format_gdp(x):
    return f"{x/1000:.1f}k" if x >= 1000 else f"{x:.0f}"

gdp_wide = (
    _gm.assign(gdp_display=_gm["gdpPercap"].map(_format_gdp))
    .pivot(index="country", columns="year", values="gdp_display")
    .reset_index()
)
gdp_wide.columns.name = None

life_wide = (
    _gm.pivot(index="country", columns="year", values="lifeExp")
    .round(1)
    .reset_index()
)
life_wide.columns.name = None

# A real region lookup, as it would actually arrive: a handful of small
# territories aren't covered at all, and two countries are spelled
# differently than in the main indicator tables.
country_region = _gm[["country", "continent"]].drop_duplicates().reset_index(drop=True)
country_region = country_region[
    ~country_region["country"].isin(["Puerto Rico", "Hong Kong, China", "Reunion"])
].reset_index(drop=True)
country_region["country"] = country_region["country"].replace({
    "Congo, Dem. Rep.": "Democratic Republic of the Congo",
    "Korea, Rep.": "South Korea",
})

gdp_wide.head()
```

```
In [ ]: life_wide.head()
```

```
In [ ]: country_region.head()
```

Part 1 — Reshaping: Wide to Long (SLO 04C)

Both `gdp_wide` and `life_wide` currently answer "what is one row about?" with *one country's entire history*. To join or analyze this data year by year, we need one row to mean *one country, in one year* — the same melt you practiced on Monday.

Task 04C.1 — Melt the GDP Table (1 pt)

Melt `gdp_wide` into long format. Every column except `"country"` should be unstacked into two new columns: `"year"` (from the column names) and `"gdp_pcap"` (from the cell values). Assign the result to `gdp_long`.

```
In [ ]: gdp_long = pd.melt( # SOLUTION
        gdp_wide, # SOLUTION
        id_vars=["country"], # SOLUTION
        var_name="year", # SOLUTION
        value_name="gdp_pcap", # SOLUTION
    ) # SOLUTION
gdp_long.head()
```

```
In [ ]: grader.check("04C.1")
```

Task 04C.2 — Melt the Life Expectancy Table (2 pts)

Now do the same for `life_wide`: melt it so that one row means *one country, in one year*, with a `"year"` column and a `"life_exp"` column. Assign the result to `life_long`.

```
In [ ]: life_long = pd.melt( # SOLUTION
        life_wide, # SOLUTION
        id_vars=["country"], # SOLUTION
        var_name="year", # SOLUTION
        value_name="life_exp", # SOLUTION
    ) # SOLUTION
life_long.head()
```

```
In [ ]: grader.check("04C.2")
```

Adjusting Data Types (given — you don't need to modify this)

Check `gdp_long.dtypes` and you'll find something odd: `year` isn't a number, and `gdp_pcap` is still text like `"5.9k"`. Both are artifacts of melting columns whose *names* were a mix of a string (`"country"`) and numbers (the years) — pandas can't assume they're all the same type. The `year` fix is one line each:

```
gdp_long["year"] = gdp_long["year"].astype(int)
life_long["year"] = life_long["year"].astype(int)
```

The `"k"` suffix in `gdp_pcap` needs a little more care — it means *thousands*, exactly as Gapminder's own website would export it:

```
In [ ]: gdp_long["year"] = gdp_long["year"].astype(int)
        life_long["year"] = life_long["year"].astype(int)

def parse_number_with_units(num):
    if not isinstance(num, str):
        return num
    if num.endswith("k"):
        return float(num[:-1]) * 1000
    return float(num)

gdp_long["gdp_pcap"] = gdp_long["gdp_pcap"].map(parse_number_with_units)
gdp_long.tail()
```

Part 2 — Keys and Relational Structure (SLO 04A)

You now have three tables: `gdp_long`, `life_long`, and `country_region`. Before joining anything, ask Wednesday's question: what column (or columns) reliably identifies "which row is this about," in a way you can check against another table?

Task 04A.1 — A Key Can Be More Than One Column (2 pts)

`country` alone does **not** uniquely identify a row of `gdp_long` — each country appears once per year. Check this two ways, using `.duplicated(subset=[...])`, which flags every row whose combination of values in those columns already appeared in an earlier row:

1. Count how many rows share their `country` value with an earlier row: `gdp_long.duplicated(subset=["country"]).sum()`. Assign it to `n_duplicate_country_rows`.
2. Count the same thing for `["country", "year"]` together: `gdp_long.duplicated(subset=["country", "year"]).sum()`. Assign it to `n_duplicate_country_year_rows`.

Which one is actually a valid key for this table — `country` alone, or the pair `["country", "year"]`?

```
In [ ]: n_duplicate_country_rows = gdp_long.duplicated(subset=["country"]).sum() # SOLUTION
        n_duplicate_country_year_rows = gdp_long.duplicated(subset=["country", "year"]).sum() # SOLUTION
        print(f'Duplicates on country alone: {n_duplicate_country_rows}')
        print(f'Duplicates on country + year: {n_duplicate_country_year_rows}')
```

```
In [ ]: grader.check("04A.1")
```

Task 04A.2 — Which Countries Won't Find a Match? (2 pts)

Before joining `gdp_long` to `country_region`, check whether every country in `gdp_long` actually has a matching row in `country_region`. Using set subtraction or `.isin()`, find every country that's in `gdp_long` but **not** in `country_region`. Assign the sorted list to `missing_from_region`.

```
In [ ]: missing_from_region = sorted(set(gdp_long["country"]) - set(country_region["country"]))  
# SOLUTION  
missing_from_region
```

```
In [ ]: grader.check("04A.2")
```

Task 04A.3 — Fix What Can Be Fixed (2 pts)

Look closely at your list from Task 04A.2. Two of those five names — "Congo, Dem. Rep." and "Korea, Rep." — are really just *spelled differently* in `country_region` (as "Democratic Republic of the Congo" and "South Korea"). The other three simply aren't in `country_region` at all.

1. Build a small lookup dictionary that renames "Democratic Republic of the Congo" → "Congo, Dem. Rep." and "South Korea" → "Korea, Rep." in `country_region`'s `country` column, the same kind of fix from Wednesday's class. Apply it with `.replace()` to a **copy** of `country_region`, and call the result `country_region_fixed`.
2. Recompute the missing-countries check from Task 04A.2 against `country_region_fixed`. Assign the new (shorter) list to `still_missing_countries`.

```
In [ ]: country_region_fixed = country_region.copy() # SOLUTION  
country_region_fixed["country"] = country_region_fixed["country"].replace({ # SOLUTION  
    "Democratic Republic of the Congo": "Congo, Dem. Rep.", # SOLUTION  
    "South Korea": "Korea, Rep.", # SOLUTION  
}) # SOLUTION  
  
still_missing_countries = sorted(set(gdp_long["country"]) - set(country_region_fixed["country"])) # SOLUTION  
still_missing_countries
```

```
In [ ]: grader.check("04A.3")
```

Part 3 — Joining (SLO 04B)

Two joins left: combine `gdp_long` and `life_long` into one table, then attach `country_region_fixed` to bring in each country's continent.

Task 04B.1 — Join GDP and Life Expectancy (2 pts)

Join `gdp_long` and `life_long` on **both** `country` and `year` — the compound key from Task 04A.1. Every row in both tables has a match (they were built from the same source), so any `how=` would technically work here — use `"inner"`. Assign the result to `gapminder`.

```
In [ ]: gapminder = pd.merge(gdp_long, life_long, on=["country", "year"], how="inner") # SOLUTION  
gapminder.head()
```

```
In [ ]: grader.check("04B.1")
```

Task 04B.2 — Attach Regions Without Losing Data (3 pts)

Join `gapminder` with `country_region_fixed` on `country`. Choose the `how=` that keeps **every** row of `gapminder`, even the countries `country_region_fixed` still doesn't cover — you don't want a garden coordinator, or a data analyst, silently losing real data because a lookup table was incomplete. Assign the result to `gapminder_with_regions`.

Then count how many rows ended up with a missing `continent`. Assign the count to `n_missing_continent`.

```
In [ ]: gapminder_with_regions = pd.merge( # SOLUTION
        gapminder, country_region_fixed, on="country", how="left" # SOLUTION
    ) # SOLUTION
n_missing_continent = gapminder_with_regions["continent"].isnull().sum() # SOLUTION
print(f'Rows with no continent: {n_missing_continent}')
gapminder_with_regions.head()
```

```
In [ ]: grader.check("04B.2")
```

Just for Fun: The Famous Gapminder Chart

(Not graded — this is the payoff for all that reshaping and joining.)

This is the animated bubble chart that made Hans Rosling's TED talks famous — the same one, built from the same data you just wrangled by hand.

```
In [ ]: px.scatter(
    gapminder_with_regions,
    x="gdp_pcap",
    y="life_exp",
    color="continent",
    animation_frame="year",
    hover_name="country",
    log_x=True,
    range_x=[100, 100000],
    range_y=[20, 90],
    labels={"gdp_pcap": "GDP per capita", "life_exp": "Life expectancy (at birth)", "continent": "Continent"},
    title="Life Expectancy vs. GDP per Capita, 1952-2007",
)
```

Look closely at what's *not* there: `px.scatter()` silently drops every row with a missing `color` value — so none of the 36 rows still missing a `continent` show up as a bubble at all, not even a gray or unlabeled one. Hong Kong, Puerto Rico, and Réunion are just... absent, every single year, with nothing in the chart to flag it. That's the same silent disappearance from Wednesday's class and this week's reading, now costing you a country in a chart instead of a row in a table.

Submission

Save your notebook, then upload the **.ipynb file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.